OBJECT ORIENTED PROGRAMMING WITH C++

ASSIGNMENT 5 – CO5

RELIABLE SOFTWARE WITH RUNTIME POLYMORPHISM AND EXCEPTIONS

Course Outcome – CO5: Build robust C++ applications by applying virtual functions, runtime polymorphism and exception handling mechanisms to develop reliable and error-resilient software solutions (virtual and pure virtual functions, runtime polymorphism, throwing, catching and rethrowing exceptions, and specifying exceptions)

Nature	:	Individual
Duration	:	One Week
Total Marks	: 50	

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- Every student must individually execute every task in this assignment, including all "Complete the Code" programs.
- Create a GitHub repository named DSA01-<RegisterNumber>-Assignment5 and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C++ source files, input files and output screenshots.
- Include a README.md with your name, register number, compiler and version (for example g++ 13), the exact g++ command used to compile (for example g++ -std=c++17 -Wall -Wextra), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. An assignment submitted without a working GitHub link will not be evaluated.

GitHub Repository Link: https://github.com/192521288/SLOT-C-OOP.git

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output with the expected output.

Predict the Output: Write the exact output of the given program before running it. Then run it and explain every difference between your prediction and the actual output, using the C++ rules involved. Predictions written after running the program will not be credited.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Assessment Components

PART A – Design: 25 Marks
PART B – Implementation & Experimental Investigation: 25 Marks

PURPOSE OF THE ASSIGNMENT

Large programs must handle many kinds of objects through a common interface, and must keep working – or fail safely – when something goes wrong.

This assignment investigates a fundamental question:

> How should a C++ programmer design a program so that new kinds of objects can be added without changing working code, and errors are reported and handled without losing data or resources?

You are expected to demonstrate the following progression:

> Abstract → Specialise → Bind at Run Time → Detect Errors → Throw → Catch → Recover → Measure → Decide

Simply obtaining a program that runs is not sufficient. Your design reasoning, experimental evidence and interpretation are all part of the assessment.

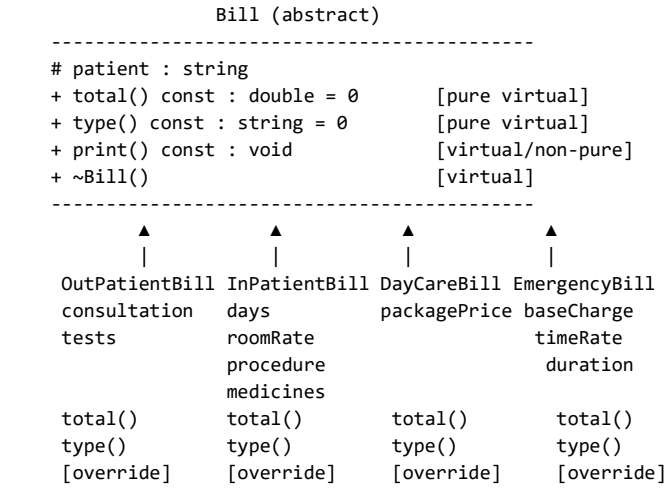
PART A – DESIGN (25 Marks)

TASK A1 – DESIGN THE POLYMORPHIC BILL HIERARCHY (7 Marks)

A1.1 Common interface

Every bill should provide total() to calculate its amount, type() to identify the bill category and print() to display the bill. Common data includes patient identity, bill/reference number and possibly date/status. The base class owns shared patient information; category-specific pricing data belongs in derived classes.

A1.2 Abstract hierarchy and UML



The pure virtual functions make Bill abstract, so a Bill object cannot be instantiated. Derived classes override both operations. print() can be implemented once in Bill and call the virtual functions.

Representative C++ interface

```
#include <iostream>
#include <string>
#include <vector>
#include <memory>

class Bill {
protected:
    std::string patient;
public:
    explicit Bill(const std::string& p) : patient(p) {}
    virtual double total() const = 0;
    virtual std::string type() const = 0;
    virtual void print() const {
        std::cout << type() << " " << patient << ": " << total() << '\n';
    }
    virtual ~Bill() = default;
};

class OutPatientBill : public Bill {
    double consultation, tests;
public:
    OutPatientBill(const std::string& p, double c, double t)
        : Bill(p), consultation(c), tests(t) {}
    double total() const override { return consultation + tests; }
    std::string type() const override { return "OP"; }
};

class InPatientBill : public Bill {
```

```
    int days;
    double roomRate, procedures, medicines;
public:
    InPatientBill(const std::string& p, int d, double r,
                  double proc, double med)
        : Bill(p), days(d), roomRate(r), procedures(proc), medicines(med) {}
    double total() const override {
        return days * roomRate + procedures + medicines;
    }
    std::string type() const override { return "IP"; }
};

class DayCareBill : public Bill {
    double packagePrice;
public:
    DayCareBill(const std::string& p, double price)
        : Bill(p), packagePrice(price) {}
    double total() const override { return packagePrice; }
    std::string type() const override { return "DAYCARE"; }
};

class EmergencyBill : public Bill {
    double baseCharge, perHour;
    int hours;
public:
    EmergencyBill(const std::string& p, double base, double rate, int h)
        : Bill(p), baseCharge(base), perHour(rate), hours(h) {}
    double total() const override { return baseCharge + perHour * hours; }
    std::string type() const override { return "EMERGENCY"; }
};
```

A1.3 Process all bills together

```
ALGORITHM CalculateDayTotal(bills, count)
BEGIN
    dayTotal ← 0
    FOR index ← 0 TO count - 1 DO
        IF bills[index] ≠ NULL THEN
            billAmount ← bills[index].total()
            dayTotal ← dayTotal + billAmount
        ENDIF
    ENDFOR
    RETURN dayTotal
END
```

The loop uses a base-class pointer for each element. Runtime dispatch selects the correct total() implementation for each concrete bill.

Alternative: one structure with a type and switch

```
enum class BillType { OP, IP, DayCare, Emergency };
struct BillRecord {
    BillType type;
    double consultation, tests, roomRate, procedures, medicines;
    int days, hours;
    double packagePrice, baseCharge, perHour;
    double total() const {
        switch (type) {
        case BillType::OP: return consultation + tests;
        case BillType::IP: return days * roomRate + procedures + medicines;
        case BillType::DayCare: return packagePrice;
        case BillType::Emergency: return baseCharge + perHour * hours;
        }
        return 0.0;
    }
};
```

Both designs process n bills in O(n) total, with O(1) dispatch/calculation per bill. A switch-based record may reserve fields unused by a particular bill. A polymorphic object generally stores only its relevant fields plus implementation-specific virtual dispatch overhead. Adding a new type changes the enum, switch and shared record/validation code; with polymorphism, add a derived class while retaining the common processing loop.

TASK A2 – UNDERSTAND RUNTIME BINDING (5 Marks)

A2.1 Early and late binding

Early binding resolves a non-virtual function using the static type known at compile time. Late binding resolves a virtual function using the object's dynamic type at run time. For Bill* b pointing to an OutPatientBill, b->total() calls OutPatientBill::total() because total() is virtual. A non-virtual call through Bill* selects the Bill implementation (if one exists) regardless of the pointed-to derived type.

A2.2 Conceptual vtable

Class	Conceptual virtual entries
Bill	destructor → Bill::~Bill; total → pure virtual slot
OutPatientBill	destructor → OutPatientBill::~OutPatientBill; total → OutPatientBill::total
InPatientBill	destructor → InPatientBill::~InPatientBill; total → InPatientBill::total

A Bill pointer refers to an object whose dynamic type determines the virtual dispatch table. The call b->total() uses the object's virtual dispatch metadata to reach the derived override. Actual vtable layout and symbols are compiler/ABI implementation details; the table above is conceptual.

A2.3 Why a virtual destructor is required

The program deletes an OPBill through a Bill*. A virtual base destructor ensures destruction begins with OPBill::~OPBill and then Bill::~Bill. Without it, deleting a derived object through a base pointer has undefined behaviour and derived cleanup may not run.

Predict the supplied output

Predicted output:

```
500 ~OPBill ~Bill
```

show() itself is non-virtual, but its body calls the virtual total(). Virtual dispatch still occurs for that inner call, so OPBill::total() returns 500. delete b invokes the derived destructor followed by the base destructor because Bill::~Bill is virtual.

If virtual is removed from ~Bill, deleting OPBill through Bill* is undefined behaviour. A common manifestation is that only ~Bill is printed and OPBill cleanup is skipped, but the exact output is not guaranteed. AddressSanitizer may report a leak or other issue depending on runtime and allocation; report the actual diagnostic rather than asserting a fixed message.

Alternative: dynamic_cast tests

If total() were non-virtual, code could test the dynamic type using dynamic_cast to each known derived type and call the matching calculation. For k bill types, this may require up to k-1 casts per bill; processing n bills can be O(nk) in this straightforward design, versus O(n) with one virtual call per bill. Virtual dispatch typically requires a vptr-sized overhead per polymorphic object; exact size is ABI-dependent. dynamic_cast also couples the processing code to every concrete bill type.

TASK A3 – DESIGN WITH ABSTRACT CLASSES (5 Marks)

A3.1 Payer interface

```
#include <algorithm>
#include <map>
#include <string>

class Payer {
public:
    virtual double amountCovered(const Bill& bill) const = 0;
    virtual std::string name() const = 0;
    virtual ~Payer() = default;
};

class SelfPay : public Payer {
public:
    double amountCovered(const Bill& bill) const override {
        return bill.total();
    }
    std::string name() const override { return "SelfPay"; }
};

class Insurance : public Payer {
    double coverLimit;
    double coPayPercent; // patient contribution, from 0 to 100
public:
    Insurance(double limit, double coPay)
        : coverLimit(limit), coPayPercent(coPay) {}
    double amountCovered(const Bill& bill) const override {
        double eligible = std::min(bill.total(), coverLimit);
        return eligible * (100.0 - coPayPercent) / 100.0;
    }
    std::string name() const override { return "Insurance"; }
};

class GovernmentScheme : public Payer {
    std::map<std::string, double> packageRates;
public:
    explicit GovernmentScheme(std::map<std::string, double> rates)
        : packageRates(std::move(rates)) {}
    double amountCovered(const Bill& bill) const override {
        auto it = packageRates.find(bill.type());
        if (it == packageRates.end())
            throw std::runtime_error("unknown package");
        return std::min(bill.total(), it->second);
    }
    std::string name() const override { return "GovernmentScheme"; }
};
```

Interpretation: SelfPay covers the complete bill. Insurance covers the eligible amount up to its policy limit after applying the stated patient co-payment percentage. GovernmentScheme uses a fixed rate keyed by bill type. The assignment does not prescribe the precise co-payment accounting convention; state the convention used and confirm it with the instructor.

A3.2 Extend with a corporate payer

```
class CorporatePayer : public Payer {
    double negotiatedPercent;
    double annualLimit;
public:
    CorporatePayer(double percent, double limit)
        : negotiatedPercent(percent), annualLimit(limit) {}
    double amountCovered(const Bill& bill) const override {
        return std::min(bill.total() * negotiatedPercent / 100.0,
                        annualLimit);
```

```
    }
    std::string name() const override { return "Corporate"; }
};
```

The new payer implements the existing Payer interface. Existing bill classes and code that calls amountCovered(const Bill&) do not need to change.

Alternative: function pointers or std::function

A policy can be stored as a callable such as std::function<double(const Bill&)> plus a separate policy-data object (for example, coverLimit). Each call remains O(1) for a fixed calculation. std::function may store a callable target and management metadata; exact space is implementation-dependent and may allocate for large callables. A class hierarchy groups policy data and behaviour in one type and supports named interfaces, while a callable design is compact for small interchangeable rules but can scatter policy state and validation.

TASK A4 – DESIGN THE EXCEPTION STRATEGY (4 Marks)

A4.1 Error catalogue and hierarchy

Error condition	Suggested exception	Handling
Negative bill charge / invalid amount	std::invalid_argument or BillingError	Reject input; show field-specific reason
Claim above insurance cover	ClaimError	Audit and reject or calculate permitted cover
Unknown government package	PackageError derived from BillingError	Log missing package code; reject claim
Missing patient record	PatientNotFound derived from BillingError	Request corrected patient identifier
Allocation failure	std::bad_alloc	Catch at application boundary; report service failure

```
#include <stdexcept>
#include <string>

class BillingError : public std::runtime_error {
public:
    using std::runtime_error::runtime_error;
};
class ClaimError : public BillingError {
public:
    using BillingError::BillingError;
};
class PackageError : public BillingError {
public:
    using BillingError::BillingError;
};
class PatientNotFound : public BillingError {
public:
    using BillingError::BillingError;
};
```

A4.2 Throwing functions and noexcept

- Validation, bill creation, payer approval, package lookup and file/database operations may throw because they can reject input, fail lookup or encounter resource errors.
- Small accessors that only return already-stored values can be noexcept if they truly cannot fail.
- Destructors should not throw. During stack unwinding, a second exception escaping a destructor can call std::terminate. Cleanup operations should handle their own failures internally.
- Do not mark a function noexcept if it calls operations that may propagate exceptions unless it catches and handles them.

A noexcept destructor is the normal C++ expectation. Use RAII members such as std::string, std::vector, std::unique_ptr and file-stream objects so cleanup occurs automatically.

Alternative: return status codes

A function could return an enum Status and write the result through an output parameter, or return a result structure containing status, value and message. Normal-path execution avoids exception machinery, while each caller must check the status. The error path also has to construct/pass the error detail. A result object requires extra storage for status and possibly a message, and every return path and caller must

propagate/check it. Exceptions keep the normal path uncluttered but are intended for exceptional failures rather than routine control flow.

TASK A5 – DESIGN RECOVERY AND RETHROWING (4 Marks)

A5.1 Where to catch

- At the validation or service boundary, catch only errors that can be meaningfully handled locally (for example, add context or translate a low-level exception).
- At the audit/logging boundary, log the error and use throw; to rethrow the same active exception without changing its dynamic type.
- At the application/controller boundary, catch known BillingError exceptions and display a safe message to the user.
- At the outermost boundary, catch std::exception and finally catch(...) to prevent an uncaught exception from terminating the process unexpectedly; log a generic internal error without exposing sensitive details.

A5.2 Stack unwinding and RAII

When an exception leaves a scope, C++ destroys fully constructed automatic objects in reverse order of construction while searching for a matching handler. RAII objects release resources in their destructors: std::vector releases its storage, std::unique_ptr deletes its owned object, and file streams close their files. This works across multiple function calls and avoids writing a separate delete/close operation in every catch block.

Predict the supplied output

Predicted output:

```
~f ~main caught 42
```

Guard g is destroyed while the exception exits f(), so ~f is printed first. Then Guard m is destroyed as stack unwinding exits the try block, so ~main is printed. The statement 'not printed' is skipped because throw transfers control to exception handling. The int handler prints caught 42.

Alternative: delete in each catch block

Manual cleanup in catch blocks can work only if every resource and every possible error path is covered. Execution time is broadly the same for the actual cleanup operations, but the code becomes larger and more fragile. Each new error path creates another opportunity to omit a delete/close or to double-release a resource. RAII centralises ownership in object lifetime and is safer.

PART B – IMPLEMENTATION & EXPERIMENTAL INVESTIGATION (25 Marks)

TASK B1 – IMPLEMENT AND VALIDATE THE POLYMORPHIC BILLING (7 Marks)

B1.1 Completed TODO 1–3

```
#include <iostream>
#include <string>

class Bill {
protected:
    std::string patient;
public:
    explicit Bill(const std::string& p) : patient(p) {}
    virtual ~Bill() = default;

    virtual double total() const = 0;           // TODO 1
    virtual std::string type() const { return "Bill"; }

    void print() const {
        std::cout << type() << ' ' << patient << ": " << total() << '\n';
    }
};

class OutPatient : public Bill {
    double consultation, tests;
public:
    OutPatient(const std::string& p, double c, double t)
        : Bill(p), consultation(c), tests(t) {}
    double total() const override { return consultation + tests; } // TODO 2
    std::string type() const override { return "OP"; }
};

class InPatient : public Bill {
    int days;
    double perDay, surgery;
public:
    InPatient(const std::string& p, int d, double rate, double s)
        : Bill(p), days(d), perDay(rate), surgery(s) {}
    double total() const override { return days * perDay + surgery; } // TODO 3
    std::string type() const override { return "IP"; }
};

int main() {
    Bill* bills[] = {
        new OutPatient("Ravi", 500, 1200),
        new InPatient("Meena", 3, 2500, 40000)
    };
    double sum = 0;
    for (Bill* b : bills) {
        b->print();
        sum += b->total();
    }
    std::cout << "Total " << sum << '\n';
    for (Bill* b : bills) delete b;
}
```

Expected output:

```
OP Ravi: 1700
IP Meena: 47500
Total 49200
```

Hand verification: outpatient = 500 + 1200 = 1700. Inpatient = 3 × 2500 + 40000 = 47500. Combined total = 1700 + 47500 = 49200.

B1.2 Compare implementation with design

The implementation follows the Part A design: Bill is abstract, total() is pure virtual, type() is virtual with a base default, print() is shared and the destructor is virtual. The two supplied concrete classes override the required operations. If the full hospital case study is implemented, add DayCareBill and EmergencyBill using the same interface.

B1.3 Remove virtual from total()

If total() is declared pure virtual but virtual is removed, the declaration is still a pure virtual function: the '= 0' syntax itself makes it virtual. C++ permits the spelling double total() const = 0; without the virtual keyword. Therefore removing only the word virtual does not remove polymorphism. To test static binding, replace the pure virtual base function with a non-virtual concrete base implementation and remove override assumptions; then a base pointer calls the base implementation. Mention this distinction in the report.

B1.4 Remove virtual destructor

Deleting an OutPatient through Bill* without a virtual base destructor is undefined behaviour. Run the altered version with AddressSanitizer and include the actual output. A clean run is not proof that the code is correct; the C++ language rule still makes this deletion undefined.

Alternative: templates / compile-time polymorphism

Use separate containers for each concrete type and a template function such as template<class Range> double sum(const Range& bills) that calls total() on each element. Each call can be statically resolved with no virtual dispatch. Each container stores its own concrete objects, but there is no single simple homogeneous list containing unrelated bill types without using a variant, type erasure or a common pointer interface. Processing separate containers requires a separate traversal per type.

TASK B2 – ENGINEER ERROR CONDITIONS (5 Marks)

Predict the supplied output

Predicted output:

```
ok done caught negative ok [log too large] caught too large
```

For 100.0, validate prints ok and process returns, then main prints done. For -5.0, validate throws invalid_argument; process catches only out_of_range, so it propagates to main, where std::exception catches it and prints caught negative. For 2e6, validate throws out_of_range; process logs it and uses throw; to rethrow the same exception. Main then catches it and prints caught too large. Only the out_of_range exception is logged because process has a handler only for that type.

Completed TODO 1 and TODO 2

```
#include <iostream>
#include <stdexcept>

class ClaimError : public std::runtime_error {
public:
    using std::runtime_error::runtime_error;
};

double approve(double billed, double cover) {
    if (billed < 0 || cover < 0)
        throw std::invalid_argument("negative amount");
    if (billed > cover)
        throw ClaimError("exceeds cover");
    return billed;
}

double settle(double billed, double cover) {
    try {
        return approve(billed, cover);
    } catch (const ClaimError& e) {
        std::cout << "audit: " << e.what() << '\n';
        throw; // rethrow the same active exception
    }
}

int main() {
    double cases[][2] = {{40000, 50000}, {60000, 50000}, {-1, 50000}};
    for (auto& c : cases) {
        try {
            double paid = settle(c[0], c[1]);
            std::cout << "approved " << paid << '\n';
        } catch (const ClaimError& e) {
            std::cout << "claim rejected: " << e.what() << '\n';
        } catch (const std::exception& e) {
            std::cout << "error: " << e.what() << '\n';
        }
    }
}
```

Expected output:

```
approved 40000
audit: exceeds cover
claim rejected: exceeds cover
error: negative amount
```

Changing throw; to throw e; throws a new exception object based on the static type of e (ClaimError in this handler), potentially slicing a more-derived exception type and resetting exception identity/metadata. throw; rethrows the currently handled exception unchanged, preserving its dynamic type.

Error injection test plan

Test	Input / trigger	Expected behaviour
Negative charge	-25	invalid_argument with clear negative-amount message
Claim exceeds cover	billed 60000, cover 50000	ClaimError; audit then reject
Unknown package	package key absent from rate map	PackageError or BillingError
Text instead of number	std::stod("abc")	invalid_argument
Allocation failure	request an intentionally impossible allocation in a controlled test	catch std::bad_alloc; do not rely on OS overcommit behaviour

For memory allocation testing, a huge allocation may terminate the process or be affected by OS overcommit and container limits. Use a controlled test environment and catch std::bad_alloc. Do not attempt destructive allocation on a production machine.

Alternative: std::optional<double>

approveOptional() could return std::optional<double>; a value indicates approval and std::nullopt indicates rejection. A richer result type is needed if the caller must know whether the reason was negative input, cover exceeded or unknown package. For one million claims, benchmark 0%, 1% and 50% invalid cases using the same inputs and optimization settings. Exception-based handling is typically efficient when errors are rare because the normal path does not branch through every error code; frequent exceptions can be much more expensive due to stack unwinding. optional/status results use ordinary control flow and are often better when rejection is expected routinely. Each optional result stores a value plus an engaged/disengaged state, typically with some representation overhead; exact size is implementation-dependent.

TASK B3 – VIRTUAL FUNCTIONS vs SWITCH vs EXCEPTIONS: THE COST (8 Marks)

B3.1 Correctness on ten bills

Use identical input values in both implementations. A small reproducible set can contain five outpatient and five inpatient bills; calculate each total by hand and compare the accumulated sums. Both implementations must return exactly the same mathematical totals. For floating-point values, compare with a tolerance rather than relying on exact binary equality.

```
// Shared test idea
double virtualSum = 0.0;
for (const Bill* b : polymorphicBills) virtualSum += b->total();

double switchSum = 0.0;
for (const BillRecord& b : records) switchSum += b.total();

if (std::abs(virtualSum - switchSum) > 1e-9)
    std::cerr << "Mismatch\n";
else
    std::cout << "Both totals agree: " << virtualSum << '\n';
```

B3.2 Benchmark 1,000 / 1,000,000 / 10,000,000 bills

The following is a benchmark harness pattern. Build separate equivalent datasets for the virtual and switch designs, warm up once, run several repetitions and report the median. Use -O2 and avoid including dataset construction in the timed region. Do not copy example timings from another computer.

```
#include <chrono>
#include <iostream>
#include <vector>
#include <memory>
#include <cstdint>

using Clock = std::chrono::steady_clock;

template<class F>
double measureMs(F fn) {
    auto start = Clock::now();
    volatile double sink = fn(); // retain computed result
    auto stop = Clock::now();
    (void)sink;
    return std::chrono::duration<double, std::milli>(stop-start).count();
}

// Repeat the equivalent total loops for each N.
// Run N = 1000, 1000000, 10000000.
// Record milliseconds and verify both sums agree before comparing.
```

Record table (fill with measured values):

Number of bills	Virtual design (ms)	Switch design (ms)	Totals agree?
1,000	_____	_____	Yes / No
1,000,000	_____	_____	Yes / No
10,000,000	_____	_____	Yes / No

B3.3 Benchmark exception vs return-code validation

Generate exactly 1,000,000 inputs for each invalid proportion: 0%, 1% and 50%. Run the same validation rules in both versions. For the exception version, throw only for invalid values and catch at the loop

boundary; for the return-code version, return a status for every input. Use the same compiler, machine, input ordering and number of repetitions.

Invalid proportion	Invalid cases	Return-code time (ms)	Exception time (ms)
0%	0	_____	_____
1%	10,000	_____	_____
50%	500,000	_____	_____

B3.4 Engineering trade-off

- Virtual functions suit open-ended runtime extension when many concrete types share a stable interface.
- A switch is straightforward for a small, closed set of types, but every new type changes central dispatch code.
- Exceptions are suitable for exceptional failures that should interrupt normal processing and propagate to a meaningful handler.
- Return codes/optional results suit routine validation outcomes, especially when invalid input is common and expected.
- Measurements must be interpreted with correctness, memory use, compiler flags and data layout; a tiny timing difference is not enough to select an architecture.

Alternative: std::variant and std::visit

```
#include <variant>
#include <vector>
#include <type_traits>

struct OutPatientBill { double consultation, tests; };
struct InPatientBill { int days; double rate, surgery; };
using AnyBill = std::variant<OutPatientBill, InPatientBill>;

double total(const AnyBill& bill) {
    return std::visit([](const auto& x) -> double {
        using T = std::decay_t<decltype(x)>;
        if constexpr (std::is_same_v<T, OutPatientBill>)
            return x.consultation + x.tests;
        else
            return x.days * x.rate + x.surgery;
    }, bill);
}
std::vector<AnyBill> bills;
```

std::variant stores one of a fixed set of alternatives in-place, with a discriminator/tag. Its element size is at least sufficient for the largest alternative plus tag/alignment overhead. std::visit dispatches among the known alternatives and can be efficient, but adding a new alternative changes the variant type and visitor logic at compile time. Benchmark 10,000,000 elements against the virtual and switch designs with the same data and compiler flags.

10,000,000 bills	Virtual (ms)	Switch (ms)	variant + visit (ms)
Measured result	_____	_____	_____

TASK B4 – FINAL ENGINEERING DECISION (5 Marks)

B4.1 Extend with a new bill and payer

Example new bill: MaternityBill, which extends Bill and implements total() and type(). Example new payer: CorporatePayer, which extends Payer and implements amountCovered() and name(). Existing processing loops use Bill* and Payer* (or smart pointers), so the extension does not require changing them.

```
class MaternityBill : public Bill {
    double deliveryPackage, medicines;
public:
    MaternityBill(const std::string& p, double package, double meds)
        : Bill(p), deliveryPackage(package), medicines(meds) {}
    double total() const override { return deliveryPackage + medicines; }
    std::string type() const override { return "MATERNITY"; }
};

class CorporatePayer : public Payer {
    double percent, maximum;
public:
    CorporatePayer(double p, double m) : percent(p), maximum(m) {}
    double amountCovered(const Bill& b) const override {
        return std::min(maximum, b.total() * percent / 100.0);
    }
    std::string name() const override { return "Corporate"; }
};
```

Add test objects to the collection and run the entire earlier test suite again. Include a table of old tests and pass/fail results after the extension.

B4.2 Resource safety

Compile the full test program with -fsanitize=address -fno-omit-frame-pointer -g. Exercise successful billing, rejected claims, unknown package handling and thrown/caught exceptions. Ensure all owned polymorphic objects use virtual destructors and RAII smart pointers. Include the actual sanitizer output. A clean run should show no detected memory errors/leaks in the tested paths; it is not a substitute for testing all ownership paths.

B4.3 Production decision

A polymorphic bill and payer architecture with exceptions is suitable for a multi-hospital system when new bill and payer types are added frequently. It isolates category-specific calculations, supports one processing interface and makes extensions local. Use smart-pointer ownership, virtual destructors, validation and structured exception handling. Exceptions should represent exceptional failures; routine claim rejection may be better represented by an explicit result/status object if invalid claims are frequent. Before production, add persistent audit logs, transactional storage, access controls, concurrency tests, monetary rounding rules, configuration validation and extensive automated tests. The final performance decision must cite the user's own benchmark results rather than assumed timings.

Alternative: configuration-driven plug-ins

A rules engine could load bill and payer rates from validated configuration files rather than compiling a new C++ class for each rate change. Each bill would interpret the applicable rule and compute its amount; runtime depends on rule lookup and evaluation, often O(1) for a direct indexed lookup or O(log r) for an ordered map with r rules. Rule storage is O(r). Configuration makes rate changes faster to deploy but

introduces parsing, schema, versioning and validation risks. New behaviour that cannot be expressed by the rule language may still require a code/plugin extension. A virtual call has O(1) dispatch overhead and behaviour is type-safe in compiled code, but adding a new class requires a software build and deployment.